B25 Attention and transformers from scratch

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2

B25 Attention and transformers from scratch 3

B25-L1 Tokens, vocabularies and embeddings 3

B25-L2 Sequence models, recurrence and its limits 8

B25-L3 Scaled dot-product attention 13

B25-L4 Masks for padding and for the future 18

B25-L5 Multi-head attention and position information 23

B25-L6 The transformer block, residuals, layer norm and feed-forward 28

Track B best-practice checklist 35

Appendix: solutions and answer keys 36

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B25 Attention and transformers from scratch

Roadmap: Months 7 to 9: NLP with deep learning. Tokens and embeddings, sequence models, attention, masks, multi-head attention, positions and the transformer block.

B25-L1 Tokens, vocabularies and embeddings

Concept

A network needs numbers. Text becomes numbers in two steps (PyTorch 2.15).

Tokenise. Split the text into tokens and map each one to an integer ID from a **vocabulary** built on training text. Production models use subword tokenisers, which split rare words into frequent pieces. This module uses whole words, so the arithmetic stays visible. Reserve ID 0 for `<pad>` and ID 1 for `<unk>`, the stand-in for words the vocabulary lacks. Pad every sequence in a batch to the same length with 0:

```
vocab = {"<pad>": 0, "<unk>": 1, "price": 2, "too": 3, "high": 4,
    "staff": 5, "great": 6}
encode("Price too high", vocab, length=5)      # [2, 3, 4, 0, 0]
encode("Staff were great", vocab, length=5)    # [5, 1, 6, 0, 0]
```

Embed. `nn.Embedding(num_embeddings, embedding_dim, padding_idx=0)` holds a weight table of shape `(vocabulary size, dim)`. Looking up ID `i` returns row `i`, the same result as multiplying a one-hot vector by the table, without building the one-hot vector. A batch of IDs shaped `(N, L)` becomes `(N, L, dim)`. Training learns the rows like any other weights (lesson B23-L3).

- IDs must be integers. Float IDs raise "Expected tensor for argument #1 'indices' to have one of the following scalar types: Long, Int".
- An ID at or beyond the vocabulary size raises "index out of range in self". It happens when IDs built with one vocabulary meet a model trained with another.
- With `padding_idx=0`, row 0 starts at zero and receives no gradient, so padding never learns a meaning.

Build the vocabulary from training text only, as with any preprocessing (lesson B10-L2), and save it with the model. Without it, the IDs mean nothing.

Why it matters

A sentiment model for open-ended answers lost twelve points of accuracy after a retrain, with no change to the model code. The retrain had rebuilt the vocabulary from new text, so "price" moved from ID 214 to ID 197, while the serving code still used the old vocabulary file. Saving the vocabulary beside the weights, and checking both carry the same version, made the mismatch impossible.

Worked example: weak code

```
1 def encode(text, vocab):
2     return [vocab[word] for word in text.split()]
```

1. **Line 2.** An unseen word raises KeyError, case is not normalised, and nothing pads or truncates.

Worked example: strong code

```
1  PAD, UNK = "<pad>", "<unk>"
2
3
4  def build_vocab(train_texts: list[str], min_count: int = 1) -> dict[str, int]:
5      """Word IDs from training text: 0 for padding, 1 for unknown words, then by frequency.
6
7      Raises:
8          ValueError: If min_count is below 1.
9      """
10     if min_count < 1:
11         raise ValueError(f"min_count must be at least 1, got {min_count}")
12     counts: dict[str, int] = {}
13     for text in train_texts:
14         for word in text.lower().split():
15             counts[word] = counts.get(word, 0) + 1
1      kept = sorted((w for w, c in counts.items() if c >= min_count), key=lambda w: (-counts[w], w))
2      return {PAD: 0, UNK: 1, **{word: i for i, word in enumerate(kept, start=2)}}
18
19
20 def encode(text: str, vocab: dict[str, int], length: int) -> list[int]:
21     """Token IDs for one text, truncated or padded to length.
22
23     Raises:
24         ValueError: If length is below 1.
25     """
26     if length < 1:
27         raise ValueError(f"length must be at least 1, got {length}")
```

```
    ids = [vocab.get(word, vocab[UNK]) for word in text.lower().split()]
[:length]
    return ids + [vocab[PAD]] * (length - len(ids))
```

1. **Line 16.** Frequent words first, ties by spelling: the same text always gives the same IDs.
2. **Line 17.** Words start at 2, so no word shares an ID with padding or unknown.
3. **Line 28.** Unseen words map to <unk> instead of raising, and long texts are cut.
4. **Line 29.** Short texts are padded to a fixed length.

Build the vocabulary on training text with reserved IDs for padding and unknown words, encode to fixed-length integer IDs, and save the vocabulary with the model.

Recap card

- **Reserved IDs in a vocabulary** 0 for <pad>, 1 for <unk>. Words start at 2. Build it on training text and save it with the model.
- **What nn.Embedding(vocab_size, dim) does** Looks up row i for ID i: (N, L) integer IDs become (N, L, dim). Same as a one-hot vector times the table.
- **padding_idx=0** Row 0 starts at zero and gets no gradient, so padding never learns a meaning.
- **Two embedding errors** Float IDs raise a dtype error. An ID at or beyond the vocabulary size raises 'index out of range in self'.

Practice

Exercise 1. Multiple choice (level 1)

Why does a vocabulary reserve an `<unk>` ID?

- a) So that words not seen in training map to one known ID instead of raising an error
- b) So that rare words get extra weight
- c) To mark the end of each sequence
- d) To fill short sequences up to a fixed length

Why?

- r1) New text always contains words the training text lacked, and every token needs a valid ID.
- r2) Unknown words are usually the most important ones.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import torch
from torch import nn

emb = nn.Embedding(10, 4, padding_idx=0)
ids = torch.tensor([[3, 5, 0, 0], [7, 1, 2, 0]])
print(tuple(emb(ids).shape), bool((emb.weight[0] == 0).all()))
```

Exercise 3. Fill the gap (level 2)

Build an embedding whose padding row never trains, and pass it integer IDs.

```
emb = nn.Embedding(len(vocab), 32, _____(1)=0)
vectors = emb(torch.tensor(batch_ids, dtype=torch._____(2)))
```

Exercise 4. Write the function (level 3)

Write `encode(text: str, vocab: dict[str, int], length: int) -> list[int]`.

- Lower-case the text and split it on whitespace.
- Map each word to its ID, or to `vocab["<unk>"]` when it is missing.
- Keep at most `length` IDs, then pad with `vocab["<pad>"]` up to `length`.
- Raise `ValueError` when `length` is below 1, or when `vocab` lacks `"<pad>"` or `"<unk>"`.

Your code must pass these checks:

- Lower-case, map unseen words to <unk>, and pad with <pad>.
- Keep at most length IDs, and pad an empty text completely.
- Raise ValueError for length below 1 or a vocabulary without <pad> and <unk>.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Token IDs for one text, truncated or padded to length.
def encode(
    text: str,
    vocab: dict[str, int],
    length: int,
) -> list[int]: ...
```

Starter code:

```
def encode(text, vocab, length):
    return [vocab[word] for word in text.split()]
```

Exercise 5. Write the function (level 3)

Write `embed(ids, table) -> np.ndarray`: what `nn.Embedding` does, in NumPy.

- `ids` is an integer array of any shape, and `table` has shape `(vocab_size, dim)`.
- Return `table[ids]`, of shape `ids.shape + (dim,)`.
- Raise `ValueError` when `ids` is not an integer array, or holds an ID below 0 or at least `vocab_size`.

What you need to know.

`arr.dtype.kind` is a one-letter code for the kind of data: `"i"` for signed integers, `"u"` for unsigned, `"f"` for floats.

```
np.array([1, 2]).dtype.kind      # 'i'
np.array([1.0]).dtype.kind       # 'f'
```

Your code must pass these checks:

- Return table[ids]: one row per ID, the same as a one-hot vector times the table.
- Raise ValueError for non-integer IDs or IDs outside 0 to vocab_size - 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Look up embedding rows by integer ID, as nn.Embedding does.
def embed(ids: np.ndarray, table: np.ndarray) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def embed(ids, table):
    return table[ids]
```

Exercise 6. Spot the bug (level 4)

Every answer that contains "affordable" is treated as padding, and "amazing" behaves like an unknown word. Tap the line responsible, then fix it.

```
def build_vocab(train_texts):
    words = sorted({word for text in train_texts for word in
    text.lower().split()})
    vocab = {word: i for i, word in enumerate(words)}
    vocab["<pad>"] = 0
    vocab["<unk>"] = 1
    return vocab
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: nn.Embedding(10, 4, padding_idx=0) has a (10, 4) weight with row 0 zero and no gradient for it; lookup equals one-hot times the table; float IDs raise 'Expected tensor for argument #1 'indices' to have one of the following scalar types: Long, Int'; ID 10 raises 'index out of range in self'. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)

B25-L2 Sequence models, recurrence and its limits

Concept

Before transformers, text models read tokens one at a time with a **recurrent network**. A hidden state `h` carries what has been read so far. At each step, `nn.RNN` (PyTorch 2.15, default `tanh`) computes `h_t = tanh(x_t @ W_ih.T + h_prev @ W_hh.T + b)`, with the same weights at every step:

```
rnn = nn.RNN(input_size=4, hidden_size=8, batch_first=True)
outputs, h_last = rnn(torch.zeros(2, 5, 4))    # (2, 5, 8) and (1, 2, 8)
```

`outputs` holds `h_t` for every step, and `h_last` the final state, with a leading dimension for the layer. `nn.LSTM` adds gates and a cell state `c`, returns `outputs, (h, c)`, and holds four times the weights: 448 against 112 for these sizes. A NumPy loop over the formula above matched `nn.RNN` to within 1e-6.

batch_first. Without `batch_first=True`, RNN and LSTM read input as `(sequence, batch, features)`. A `(2, 5, 4)` batch then means five sequences of length 2, and the output still has shape `(2, 5, 8)`: no error, the wrong model. Set `batch_first=True` and assert shapes.

Two limits led to attention:

- **Long-range memory.** Gradients flow back through every step and are multiplied by a factor at each. Factors below 1 shrink the signal: 0.9 over 50 steps leaves 0.0052. Factors above 1 blow it up: 1.1 over 50 steps gives 117.4. LSTM gates slow the shrinking, but a word 200 tokens back still reaches the output faintly.
- **No parallelism.** Step `t` needs step `t - 1`, so 512 tokens take 512 steps, one after another. Attention (lesson B25-L3) relates every pair of positions in one matrix product.

A recurrent model is still a fair baseline for short sequences. Lessons B25-L3 to B25-L6 build what replaced it.

Why it matters

A team classified long complaint transcripts with an LSTM that read the text from start to end. Accuracy was good on short complaints and near chance on long ones, where the reason for the complaint came in the first lines. Error analysis by transcript length (lesson B12-L3) showed the pattern. An attention model, which compares every position with every other directly, kept its accuracy at all lengths.

Worked example: weak code

```
lstm = nn.LSTM(input_size=32, hidden_size=64)
outputs, (h, c) = lstm(embedded)       # embedded: (batch, length, 32)
logits = head(outputs[:, -1])
```

1. **Line 1.** batch_first defaults to False: the batch is read as the sequence, with no error.
2. **Line 3.** Takes the last step of a padded batch, which for short texts is a padding step.

Worked example: strong code

```
import numpy as np


def rnn_forward(
    x: np.ndarray, W_ih: np.ndarray, W_hh: np.ndarray, b: np.ndarray
) -> np.ndarray:
    """Hidden states of a tanh RNN over one sequence, as nn.RNN computes them.

    Args:
        x: One sequence, shape (length, input_size).
        W_ih: Input weights, shape (hidden_size, input_size).
        W_hh: Recurrent weights, shape (hidden_size, hidden_size).
        b: Combined bias, shape (hidden_size,).

    Returns:
        Hidden state after each step, shape (length, hidden_size).

    Raises:
        ValueError: If the shapes do not fit together.
    """
    if x.ndim != 2 or W_ih.shape[1] != x.shape[1] or W_hh.shape != (len(b), len(b)):
        raise ValueError(f"shapes do not fit: x {x.shape}, W_ih {W_ih.shape}, W_hh {W_hh.shape}")
    h = np.zeros(len(b))
    states = []
    for x_t in x:
```

```
        h = np.tanh(x_t @ W_ih.T + h @ W_hh.T + b)
        states.append(h)
    return np.stack(states) if states else np.zeros((0, len(b)))
```

1. **Line 23.** The state starts at zero, as in nn.RNN without an initial state.
2. **Line 25.** One step at a time: step t cannot start before step t - 1.
3. **Line 26.** The same weights at every step, mixing the new input with the previous state.

A recurrent network carries a hidden state step by step. Set batch_first=True, and remember its two limits: fading long-range signal and no parallelism.

Recap card

- **The RNN step** h_t = tanh(x_t @ W_ih.T + h_prev @ W_hh.T + b), with the same weights at every step.
- **nn.LSTM outputs with batch_first=True** outputs (batch, length, hidden), plus h and c of shape (layers, batch, hidden).
- **batch_first default** False for RNN and LSTM: (length, batch, features). Wrong layouts often run without error. Set it and assert shapes.
- **Two limits of recurrent models** Gradients fade or explode over long sequences (0.9 ** 50 = 0.0052), and steps cannot run in parallel.

Practice

Exercise 1. Multiple choice (level 1)

Why does a plain RNN struggle to use a word 200 tokens before the end of a text?

- a) The gradient reaching that word is multiplied by a factor at every step, and products of factors below 1 shrink towards zero
- b) RNNs can only read 100 tokens
- c) The embedding table forgets early words
- d) The hidden state is reset every 50 tokens

Why?

- r1) Training signal for early words passes back through every step, so it fades: 0.9 over 50 steps is 0.0052.
- r2) Early words are less important by nature.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import torch
```

```
from torch import nn

lstm = nn.LSTM(4, 8, batch_first=True)
outputs, (h, c) = lstm(torch.zeros(2, 5, 4))
print(tuple(outputs.shape), tuple(h.shape), tuple(c.shape))
```

Exercise 3. Fill the gap (level 2)

Read batches as `(batch, length, features)`, and complete one step of the recurrence in NumPy.

```
lstm = nn.LSTM(32, 64, _____(1)=True)

h = np._____(2)(x_t @ W_ih.T + h @ W_hh.T + b)
```

Exercise 4. Write the function (level 3)

Write `rnn_forward(x, W_ih, W_hh, b) -> np.ndarray`: the hidden states of a tanh RNN over one sequence, starting from a zero state.

- At each step, `h = tanh(x_t @ W_ih.T + h @ W_hh.T + b)`.
- Return the states stacked, shape `(length, hidden_size)`.
- Raise `ValueError` when `x` is not 2-D, `W_ih` does not have `x`'s feature count as columns, or `W_hh` is not `(hidden_size, hidden_size)` with `hidden_size = len(b)`.

What you need to know.

`np.stack(list_of_arrays)` joins equal-shaped arrays along a new first axis:

```
np.stack([np.array([1, 2]), np.array([3, 4])])    # array([[1, 2], [3, 4]])
```

Your code must pass these checks:

- Carry h from step to step: h = tanh(x_t @ W_ih.T + h @ W_hh.T + b), starting at zeros.
- Use the previous state at each step, so the order of the inputs changes the result.
- Raise ValueError when the shapes do not fit.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Hidden states of a tanh RNN over one sequence, as nn.RNN computes them.
def rnn_forward(
    x: np.ndarray,
    W_ih: np.ndarray,
    W_hh: np.ndarray,
    b: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```numpy
import numpy as np


def rnn_forward(x, W_ih, W_hh, b):
    return np.tanh(x @ W_ih.T + b)
```

Exercise 5. Predict the output (level 3)

A gradient passes back through 50 steps and is multiplied by the same factor at each. What does this print?

```
print(round(0.9 ** 50, 4), round(1.1 ** 50, 1))
```

Exercise 6. Spot the bug (level 4)

The model trains without errors, but its accuracy stays at chance. Each batch of embedded answers has shape `(64, 30, 32)`: batch, length, features. Tap the line responsible, then fix it.

```pytorch
from torch import nn


class AnswerClassifier(nn.Module):
    def __init__(self, vocab_size, n_classes):
        super().__init__()
        self.tokens = nn.Embedding(vocab_size, 32, padding_idx=0)
        self.lstm = nn.LSTM(32, 64)
        self.head = nn.Linear(64, n_classes)

    def forward(self, ids):
        outputs, (h, c) = self.lstm(self.tokens(ids))
        return self.head(h[-1])
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: nn.RNN(4, 8, batch_first=True) on (2, 5, 4) gives outputs (2, 5, 8) and h (1, 2, 8), with tanh by default and 112 parameters; nn.LSTM gives (2, 5, 8), (1, 2, 8), (1, 2, 8) and 448 parameters; without batch_first the same input runs and returns (2, 5, 8); a NumPy tanh loop with W_ih, W_hh and b_ih + b_hh matches nn.RNN within 1e-6. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)

B25-L3 Scaled dot-product attention

Concept

Attention lets every position look at every other position directly. Each token's vector is projected three ways: a **query**, what this position looks for, a **key**, what it offers, and a **value**, what it passes on. For queries `Q`, keys `K` and values `V`, each of shape `(length, d)`:

```
scores = Q @ K.T / np.sqrt(d)              # (length, length)
weights = softmax(scores, axis=-1)         # each row sums to 1
output = weights @ V                       # (length, d)
```

Row `i` of `weights` says how much position `i` draws from each position, and output `i` is that weighted average of the values.

Why divide by `sqrt(d)`. For random vectors with unit variance, the dot product's standard deviation grows like `sqrt(d)`: about 2 at `d = 4`, 8 at 64 and 22 at 512, in a simulation of 20,000 pairs. Large scores push softmax towards one 1 and zeros elsewhere, where its gradients are almost zero. Dividing by `sqrt(d)` brings the spread back to about 1.

A stable softmax. `exp(1000)` overflows, so the textbook softmax of `[1000, 1001, 1002]` returns `nan`. Subtract each row's maximum first. Softmax does not change when you add a constant to every input, and the largest exponent becomes `exp(0) = 1`. The result is `[0.09, 0.2447, 0.6652]`.

A small example: the query `[1, 0]` against keys `[1, 0]`, `[0, 1]` and `[1, 1]` scores 0.71, 0 and 0.71 after scaling. The weights are 0.4011, 0.1978 and 0.4011, so with values 10, 20 and 30 the output is 20.

PyTorch 2.15 computes this with `torch.nn.functional.scaled_dot_product_attention(q, k, v)` on tensors shaped `(..., length, d)`. A NumPy version matched it to within 1e-5. The projections that make Q, K and V are learned `nn.Linear` layers (lesson B25-L5).

Why it matters

A team wrote its own attention layer for a text classifier, and training produced `nan` loss within the first epoch on long answers. Long inputs made a few scores huge, and the softmax divided infinity by infinity. Subtracting the row maximum before `exp` fixed it, and the scaled scores also trained faster, with no change to the rest of the model.

Worked example: weak code

```
def attention(Q, K, V):
    weights = np.exp(Q @ K.T)
    weights = weights / weights.sum()
    return weights @ V
```

1. **Line 2.** No scaling and no max subtraction: overflows to inf, then nan.
2. **Line 3.** Normalises over the whole matrix, so rows do not each sum to 1.

Worked example: strong code

```
import numpy as np


def softmax(x: np.ndarray, axis: int = -1) -> np.ndarray:
    """Softmax along an axis, with the maximum subtracted for stability."""
    shifted = x - x.max(axis=axis, keepdims=True)
    e = np.exp(shifted)
    return e / e.sum(axis=axis, keepdims=True)


def attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Scaled dot-product attention for one sequence.

    Args:
        Q: Queries, shape (n_queries, d).
        K: Keys, shape (n_keys, d).
        V: Values, shape (n_keys, d_v).

    Returns:
        Outputs (n_queries, d_v) and weights (n_queries, n_keys).

    Raises:
        ValueError: If Q and K differ in width or K and V in length.
    """
    if Q.shape[-1] != K.shape[-1] or len(K) != len(V):
        raise ValueError(f"shapes do not fit: Q {Q.shape}, K {K.shape}, V {V.shape}")
    weights = softmax(Q @ K.T / np.sqrt(Q.shape[-1]), axis=-1)
    return weights @ V, weights
```

1. **Line 6.** The largest value becomes 0, so exp never overflows.
2. **Line 8.** keepdims keeps the axis, so each row divides by its own sum.
3. **Line 27.** Scale by sqrt(d), then softmax across the keys of each query.
4. **Line 28.** Return the weights too: they show what each position attended to.

Scores are Q @ K.T / sqrt(d), softmax runs across keys with the row maximum subtracted, and the output is the weighted average of the values.

Recap card

- **Scaled dot-product attention** weights = softmax(Q @ K.T / sqrt(d)) across keys; output = weights @ V.
- **Why scale by sqrt(d)** Dot products grow like sqrt(d): about 2, 8 and 22 at d = 4, 64, 512. Scaling keeps softmax out of saturation.
- **Stable softmax** Subtract the maximum along the axis with keepdims=True before exp, then divide by the sum along the same axis.
- **Attention in PyTorch 2.15** torch.nn.functional.scaled_dot_product_attention(q, k, v) on (..., length, d) tensors.

Practice

Exercise 1. Multiple choice (level 1)

Why does attention divide the scores by `sqrt(d)`?

- a) Dot products of d-dimensional vectors grow with d. Large scores saturate softmax and its gradients vanish
- b) It makes each row of weights sum to 1
- c) It stops exp from overflowing on any input
- d) It normalises the values V

Why?

- r1) The standard deviation of a dot product of unit-variance vectors is about sqrt(d), so dividing by it keeps scores near unit scale.
- r2) Square roots are cheap to compute.

Exercise 2. Predict the output (level 2)

What does this print?

```
import numpy as np

q = np.array([1.0, 0.0])
K = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
V = np.array([10.0, 20.0, 30.0])
scores = K @ q / np.sqrt(2)
w = np.exp(scores - scores.max())
w = w / w.sum()
print(w.round(4).tolist(), round(float(w @ V), 4))
```

Exercise 3. Fill the gap (level 2)

Complete the scaled scores and the stable shift before `exp`.

```
scores = Q @ K.T / np._____(1)(d)
e = np.exp(scores - scores._____(2)(axis=-1, keepdims=True))
```

Exercise 4. Write the function (level 3)

Write `softmax(x, axis=-1) -> np.ndarray`, stable for large inputs.

- Subtract the maximum along `axis` (keeping the axis) before `np.exp`.
- Divide by the sum along the same axis, so values along it sum to 1.

What you need to know.

`keepdims=True` keeps the reduced axis with length 1, so the result broadcasts back against the original array:

```
a = np.array([[1.0, 3.0], [2.0, 6.0]])
a.max(axis=1, keepdims=True)        # array([[3.], [6.]])
a - a.max(axis=1, keepdims=True)    # array([[-2., 0.], [-4., 0.]])
```

Your code must pass these checks:

- Subtract the maximum before exp, so large inputs do not overflow.
- Normalise each row separately along the last axis.
- Use the axis argument for both the maximum and the sum, with keepdims=True.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Softmax along an axis, with the maximum subtracted for stability.
def softmax(x: np.ndarray, axis: int = -1) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def softmax(x, axis=-1):
    e = np.exp(x)
    return e / e.sum()
```

Exercise 5. Write the function (level 3)

Write `attention(Q, K, V) -> tuple[np.ndarray, np.ndarray]` for one sequence. A stable `softmax(x, axis=-1)` is defined for you.

- `Q` is `(n_queries, d)`, `K` is `(n_keys, d)` and `V` is `(n_keys, d_v)`.
- Weights are `softmax(Q @ K.T / sqrt(d), axis=-1)`. Return `(weights @ V, weights)`.
- Raise `ValueError` when `Q` and `K` differ in width, or `K` and `V` in length.

Your code must pass these checks:

- Weights are softmax(Q @ K.T / sqrt(d)) across keys, and the output is weights @ V.
- Each query's weights sum to 1 across the keys.
- Raise ValueError when Q and K differ in width or K and V in length.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Scaled dot-product attention for one sequence.
def attention(
    Q: np.ndarray,
    K: np.ndarray,
    V: np.ndarray,
) -> tuple[np.ndarray, np.ndarray]: ...
```

Starter code:

```python
import numpy as np


def attention(Q, K, V):
    return Q @ K.T @ V, None
```

Exercise 6. Spot the bug (level 4)

Plotting the attention weights shows each column summing to 1, not each row. Tap the line responsible, then fix it.

```python
import numpy as np


def attention(Q, K, V):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    weights = weights / weights.sum(axis=0, keepdims=True)
    return weights @ V, weights
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: torch.nn.functional.scaled_dot_product_attention(query, key, value, attn_mask=None, dropout_p=0.0, is_causal=False, scale=None, enable_gqa=False) matches a NumPy softmax(QK^T/sqrt(d))V within 1e-5; the naive softmax of [1000, 1001, 1002] is nan and the stable one [0.09, 0.2447, 0.6652]; simulated dot-product standard deviations are 1.98, 7.98 and 22.46 at d = 4, 64 and 512, and about 1 after scaling.

(verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU) and NumPy, November 2026. Signature read from the function's docstring.)

B25-L4 Masks for padding and for the future

Concept

Two kinds of position must stay out of attention.

Padding. Padded positions (lesson B25-L1) carry no text. Set their scores to minus infinity before softmax, so their weights become exactly 0 and every query ignores them.

The future. A model that predicts the next token must not read it. A **causal mask** lets position `i` attend to positions 0 to `i` only: keep the lower triangle, `np.tril(np.ones((L, L), dtype=bool))`. Row 0 then puts all its weight on itself.

```
scores = np.where(allowed, scores, -np.inf)    # allowed: True where attention may look
```

Two opposite conventions in PyTorch 2.15:

- `F.scaled_dot_product_attention(q, k, v, attn_mask=allowed)` takes a boolean mask where `True` means take part. `is_causal=True` builds the causal mask for you, and gave the same outputs as the lower-triangle mask.
- `nn.MultiheadAttention(...)(q, k, v, key_padding_mask=is_pad)` takes a mask where `True` means ignore this key. In a check, padded keys received weight 0.0.

Passing one function's mask to the other inverts it, with no error. Name each mask by its meaning, `allowed` or `is_pad`, and flip it with `~` where needed.

Test it. A causal model's output at position `t` must not change when a later token changes. In a check, editing the last position's value left every earlier output unchanged. Padding passes the same test: adding pads must not change the outputs at real positions.

Every query needs an allowed key. In NumPy, a row with every key masked gives `nan`. `scaled_dot_product_attention` on a CPU returned zeros for such rows, a quieter failure. Pad on the right and keep each position allowed to see itself.

Why it matters

A model that drafted the next word of a survey answer reached a near-perfect validation loss within an hour, then produced nonsense when used. Its attention had no causal mask, so during training every position could read the word it was asked to predict. The test "change a later token, check earlier outputs stay the same" would have failed on the first run.

Worked example: weak code

```
pad = ids == 0
out = F.scaled_dot_product_attention(q, k, v, attn_mask=pad)
```

1. **Line 2.** attn_mask True means take part, so this attends only to padding and ignores the text.

Worked example: strong code

```
import numpy as np


def causal_padding_mask(ids: np.ndarray, pad_id: int = 0) -> np.ndarray:
    """Allowed attention for one right-padded sequence: past or present, never padding.

    Args:
        ids: Token IDs, shape (length,), padding only at the end.
        pad_id: The padding ID.

    Returns:
        Boolean (length, length): True where query i may attend to key j.

    Raises:
        ValueError: If ids is not 1-D or starts with padding.
    """
    if ids.ndim != 1 or len(ids) == 0 or ids[0] == pad_id:
        raise ValueError("need a non-empty 1-D sequence that starts with a real token")
    causal = np.tril(np.ones((len(ids), len(ids)), dtype=bool))
    return causal & (ids != pad_id)[None, :]


def masked_attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray, allowed: np.ndarray) -> np.ndarray:
    """Attention where allowed is False gets zero weight."""
    scores = np.where(allowed, Q @ K.T / np.sqrt(Q.shape[-1]), -np.inf)
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    return (weights / weights.sum(axis=-1, keepdims=True)) @ V
```

1. **Line 17.** With a real first token, every query has at least one allowed key.

2. **Line 19.** Lower triangle: each position sees itself and the past.
3. **Line 20.** Combine with the padding mask, applied to keys (columns).
4. **Line 25.** Masked scores become minus infinity, so their weights are exactly 0.

Mask padding and the future by setting their scores to minus infinity. Know which convention each PyTorch function uses, and test that later tokens and padding cannot change earlier outputs.

Recap card

- **Masking in attention** Set masked scores to minus infinity before softmax, so their weights are exactly 0.
- **Causal mask** np.tril(np.ones((L, L), dtype=bool)): position i sees 0 to i. F.scaled_dot_product_attention(..., is_causal=True) builds it.
- **PyTorch 2.15 mask conventions** scaled_dot_product_attention attn_mask: True = take part. MultiheadAttention key_padding_mask: True = ignore.
- **Mask tests** Changing a later token, or adding padding, must not change earlier outputs or outputs at real positions.

Practice

Exercise 1. Multiple choice (level 1)

A model is trained to predict each next word of an answer. Which mask does its attention need?

- a) A causal mask, so position i sees positions 0 to i only, plus masking of padding
- b) Only a padding mask
- c) No mask: attention learns to ignore the future
- d) A mask that hides the current word

Why?

- r1) Without the causal mask, each position can read the word it must predict, and the model learns to copy it.
- r2) Masks make training faster.

Exercise 2. Predict the output (level 2)

Three positions with equal scores and a causal mask. What does this print?

```
1 import numpy as np
2
3 scores = np.zeros((3, 3))
```

```
allowed = np.tril(np.ones((3, 3), dtype=bool))
masked = np.where(allowed, scores, -np.inf)
e = np.exp(masked - masked.max(axis=-1, keepdims=True))
print((e / e.sum(axis=-1, keepdims=True)).round(4).tolist())
```

Exercise 3. Fill the gap (level 2)

Build a causal mask and apply it to the scores.

```
allowed = np._____(1)(np.ones((L, L), dtype=bool))
scores = np.where(allowed, scores, -np._____(2))
```

Exercise 4. Write the function (level 3)

Write `masked_attention(Q, K, V, allowed) -> np.ndarray`.

- Scores are `Q @ K.T / sqrt(d)`. Where `allowed` is `False`, set the score to `-np.inf`.
- Apply a stable softmax across keys and return `weights @ V`.
- Raise `ValueError` when `allowed` is not `(n_queries, n_keys)`, or when a row has no allowed key.

What you need to know.

`arr.any(axis=1)` is `True` for each row with at least one `True`, and `.all()` then checks every row:

```
m = np.array([[True, False], [False, False]])
m.any(axis=1)          # array([ True, False])
m.any(axis=1).all()    # False
```

Your code must pass these checks:

- Masked keys must get weight 0: earlier outputs cannot depend on later values.
- With every key allowed, return plain scaled dot-product attention.
- Raise ValueError for a mask of the wrong shape or a row with no allowed key.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Scaled dot-product attention where allowed is False gets zero weight.
def masked_attention(
    Q: np.ndarray,
    K: np.ndarray,
    V: np.ndarray,
    allowed: np.ndarray,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np
```

```

def masked_attention(Q, K, V, allowed):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    e = np.exp(scores - scores.max(axis=-1, keepdims=True))
    return (e / e.sum(axis=-1, keepdims=True)) @ V
```

Exercise 5. Write the function (level 3)

Write `causal_padding_mask(ids, pad_id=0) -> np.ndarray` for one right-padded sequence of token IDs.

- Return a boolean `(L, L)` array: `True` where query `i` may attend to key `j`, that is `j <= i` and `ids[j]` is not padding.
- Raise `ValueError` when `ids` is not 1-D, is empty, or starts with `pad_id`.

Your code must pass these checks:

- Allow j <= i, and never a padding key.
- Without padding it is the lower triangle. Respect the pad_id argument.
- Raise ValueError for a sequence that is not 1-D, is empty, or starts with padding.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Allowed attention for one right-padded sequence: past or present, never padding.
def causal_padding_mask(
    ids: np.ndarray,
    pad_id: int = 0,
) -> np.ndarray: ...
```

Starter code:

```
import numpy as np


def causal_padding_mask(ids, pad_id=0):
    return np.ones((len(ids), len(ids)), dtype=bool)
```

Exercise 6. Spot the bug (level 4)

Predictions ignore the words of each answer and depend only on how much padding it has. Tap the line responsible, then fix it.

```
from torch import nn

attn = nn.MultiheadAttention(embed_dim=64, num_heads=4, batch_first=True)


def encode(x, ids):
    real = ids != 0
```

```
    out, _ = attn(x, x, x, key_padding_mask=real)
    return out
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: scaled_dot_product_attention with a boolean lower-triangle attn_mask equals is_causal=True; True means take part; editing the last position leaves earlier causal outputs unchanged; nn.MultiheadAttention gives padded keys weight 0.0 when key_padding_mask is True there; a fully masked row returned zeros from scaled_dot_product_attention on CPU, and nan in a NumPy softmax. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU) and NumPy, November 2026.)

B25-L5 Multi-head attention and position information

Concept

One attention layer gives each query one weighting over the keys. **Multi-head attention** runs `h` smaller attentions side by side, so one head can follow the subject while another follows a negation (PyTorch 2.15):

1. Project the input `X` of shape `(N, L, d)` with three learned `nn.Linear(d, d)` layers into Q, K and V.
2. Split `d` into `h` heads of `d // h`: `x.reshape(N, L, h, d // h).transpose(1, 2)` gives `(N, h, L, d // h)`.
3. Attend within each head, as in lesson B25-L3.
4. Merge: transpose back, reshape to `(N, L, d)`, and apply one more `nn.Linear(d, d)`.

`nn.MultiheadAttention(embed_dim=d, num_heads=h, batch_first=True)` does all four. It holds `4 * d * d + 4 * d` parameters whatever the number of heads: 288 for `d = 8` with 2 heads or 4, and 1,050,624 for `d = 512`. A `d` that does not divide by `h` raises "embed_dim must be divisible by num_heads".

Attention cannot see order. Shuffle the input tokens and the outputs come back shuffled in the same way, with the same values: attention compares contents, not places. In a check, permuting five tokens permuted the outputs exactly. "not good" and "good not" look identical. So add position information to the embeddings before the first layer:

- **Learned positions:** `nn.Embedding(max_len, d)` indexed by positions 0 to `L - 1` and added to the token embeddings. It cannot go beyond `max_len`.

- **Sinusoidal positions** (Vaswani and others, 2017): `PE[pos, 2i] = sin(pos / 10000 ** (2i / d))` and `PE[pos, 2i + 1] = cos(...)` at the same angle. No weights, and defined for any position, though the model has not trained on positions beyond its longest training text. For `d = 4`, position 1 is `[0.8415, 0.5403, 0.01, 1.0]`.

Why it matters

A transformer classifier of complaints treated "not resolved" and "resolved, not" alike and missed many unresolved cases. Its embeddings had no position information: the team had dropped the positional layer while tidying the model code, and nothing failed. A test that shuffles the tokens and expects a different prediction would have caught it at once.

Worked example: weak code

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, d):
        super().__init__()
        self.tokens = nn.Embedding(vocab_size, d, padding_idx=0)
        self.attn = nn.MultiheadAttention(d, num_heads=5)

    def forward(self, ids):
        x = self.tokens(ids)
        return self.attn(x, x, x)[0]
```

1. **Line 5.** No batch_first, and d must divide by 5: with d = 64 this raises.
2. **Line 8.** No positions: word order is invisible to everything after this line.

Worked example: strong code

```
import numpy as np
import torch
from torch import nn


def sinusoidal_positions(n: int, d: int) -> np.ndarray:
    """Sinusoidal position encodings, shape (n, d), with d even."""
    pos = np.arange(n)[:, None]
    angle = pos / 10000 ** (np.arange(0, d, 2)[None, :] / d)
    pe = np.zeros((n, d))
    pe[:, 0::2], pe[:, 1::2] = np.sin(angle), np.cos(angle)
    return pe


class Encoder(nn.Module):
    """Token embeddings plus learned positions, then multi-head self-attention."""

```

```
18      def __init__(self, vocab_size: int, d: int, heads: int, max_len: int)
    -> None:
19          super().__init__()
20          if d % heads:
3               raise ValueError(f"d={d} must divide by heads={heads}")
22          self.tokens = nn.Embedding(vocab_size, d, padding_idx=0)
23          self.positions = nn.Embedding(max_len, d)
24          self.attn = nn.MultiheadAttention(d, heads, batch_first=True)
25
26      def forward(self, ids: torch.Tensor) -> torch.Tensor:
27          where = torch.arange(ids.shape[1], device=ids.device)
4           x = self.tokens(ids) + self.positions(where)
5           return self.attn(x, x, x, key_padding_mask=ids == 0)[0]
```

1. **Line 9.** One frequency per pair of columns, from fast to slow.
2. **Line 11.** Sine in even columns, cosine in odd columns.
3. **Line 21.** Fail early with a clear message.
4. **Line 28.** Each token vector now carries its position.
5. **Line 29.** Padding is ignored: True in key_padding_mask means ignore (lesson B25-L4).

Multi-head attention splits d into heads, attends in each and merges them. Attention ignores order, so add learned or sinusoidal positions to the embeddings.

Recap card

- **Multi-head attention steps** Project to Q, K, V; split d into h heads of d // h; attend per head; merge and project with Linear(d, d).
- **nn.MultiheadAttention parameters** 4 * d * d + 4 * d, whatever the number of heads. d must divide by num_heads.
- **Why positions** Attention ignores order: shuffled inputs give the same outputs, shuffled. Add learned or sinusoidal positions to the embeddings.
- **Sinusoidal positions** PE[pos, 2i] = sin(pos / 10000 ** (2i / d)), PE[pos, 2i + 1] = cos of the same angle.

Practice

Exercise 1. Multiple choice (level 1)

Why does a transformer add position information to its token embeddings?

- a) Attention compares contents only, so without positions a shuffled sentence gives the same outputs, shuffled
- b) To make the embeddings larger
- c) To mark which tokens are padding

- d) Because softmax needs positive inputs

Why?

- r1) Without positions, "not good" and "good not" are the same set of vectors to attention.
- r2) Positions speed up the matrix products.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```python
import torch
from torch import nn

x = torch.zeros(2, 5, 8)
heads = x.reshape(2, 5, 2, 4).transpose(1, 2)
two = sum(p.numel() for p in nn.MultiheadAttention(8, 2,
    batch_first=True).parameters())
four = sum(p.numel() for p in nn.MultiheadAttention(8, 4,
    batch_first=True).parameters())
print(tuple(heads.shape), two, four)
```

Exercise 3. Fill the gap (level 2)

Split a `(N, L, d)` tensor into heads, and build attention that reads `(batch, length, features)`.

```python
heads = x.reshape(N, L, h, d // h)._____(1)(1, 2)
attn = nn.MultiheadAttention(64, 4, _____(2)=True)
```

Exercise 4. Write the function (level 3)

Write `split_heads(x, n_heads) -> np.ndarray` for one sequence.

- `x` has shape `(L, d)`. Return shape `(n_heads, L, d // n_heads)`, where head `k` holds columns `k * dh` to `(k + 1) * dh - 1`.
- Raise `ValueError` when `x` is not 2-D, `n_heads` is below 1, or `d` does not divide by `n_heads`.

What you need to know.

In NumPy, `arr.transpose(1, 0, 2)` reorders axes, like PyTorch's `permute`: the new axis 0 is the old axis 1.

```python
a = np.zeros((5, 2, 4))
a.transpose(1, 0, 2).shape    # (2, 5, 4)
```

Your code must pass these checks:

- Head k takes columns k * dh to (k + 1) * dh - 1 of every row.
- Reshape to (L, h, dh), then move the head axis first.

- Raise ValueError for non-2-D input, heads below 1, or a width that does not divide.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Split the width of one sequence into attention heads.
def split_heads(x: np.ndarray, n_heads: int) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def split_heads(x, n_heads):
    return x.reshape(n_heads, x.shape[0], -1)
```

Exercise 5. Write the function (level 3)

Write `sinusoidal_positions(n, d) -> np.ndarray`, shape `(n, d)`.

- For position `pos` and `i` in `0, 2, 4, ...`: `angle = pos / 10000 ** (i / d)`. Column `i` holds `sin(angle)` and column `i + 1` holds `cos(angle)`.
- Raise `ValueError` when `n` is below 1 or `d` is not a positive even number.

Your code must pass these checks:

- Even columns sin(pos / 10000 ** (i / d)), odd columns cos of the same angle.
- Each sin and cos pair satisfies sin^2 + cos^2 = 1, so every row's squares sum to d / 2.
- Raise ValueError for n below 1 or d that is not a positive even number.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Sinusoidal position encodings.
def sinusoidal_positions(n: int, d: int) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def sinusoidal_positions(n, d):
    return np.zeros((n, d))
```

Exercise 6. Spot the bug (level 4)

After merging heads, each token's vector holds pieces of other tokens' vectors. `merge_heads` should undo `split_heads`. Tap the line responsible, then fix it.

```python
import numpy as np


def merge_heads(heads):
    n_heads, length, dh = heads.shape
    return heads.reshape(length, n_heads * dh)
```

Write down the line number of the bug. Then write the fix.

Sources

- PyTorch 2.15: nn.MultiheadAttention(8, h) has 288 parameters for h = 2 and 4, and 1,050,624 for d = 512; embed_dim not divisible by num_heads raises 'embed_dim must be divisible by num_heads'; without positions, permuting the input permutes the output exactly; reshape and transpose(1, 2) give (2, 2, 5, 4). (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)
- The sinusoidal encoding PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d)) comes from Vaswani and others, Attention Is All You Need (2017). (search-only: Search results citing arXiv 1706.03762, November 2026. arxiv.org was blocked in the build session. Values for d = 4 computed locally.)

B25-L6 The transformer block, residuals, layer norm and feed-forward

Concept

A transformer stacks identical **blocks**. Each block has two sub-layers, and each sub-layer sits inside a residual connection with a layer norm. The pre-norm form (PyTorch 2.15):

```
x = x + attention(norm_1(x))        # tokens exchange information
x = x + feed_forward(norm_2(x))     # each token is transformed on its own
```

- **Residual connections**, the `x + ...`, let each sub-layer add a correction to its input, and let gradients pass straight through deep stacks.
- **Layer norm** rescales each token's vector to mean 0 and standard deviation 1 across its features, then applies a learned scale and shift. `nn.LayerNorm(4)` turns `[1, 2, 3, 4]` into `[-1.3416, -0.4472, 0.4472, 1.3416]`. It works per token, not across the batch, so it behaves the same in training and at prediction time.
- **Feed-forward**: `nn.Linear(d, 4 * d)`, an activation, then `nn.Linear(4 * d, d)`, applied to each position separately.

`nn.TransformerEncoderLayer(d_model, nhead)` builds one block. Check three defaults in PyTorch 2.15:

- `batch_first=False`: a `(batch, length, d)` input is read as `(length, batch, d)`, and the output has the same shape, so nothing fails (lesson B25-L2).
- `norm_first=False`: the original post-norm order. Pass `norm_first=True` for the pre-norm form above.
- `dim_feedforward=2048`: with `d_model=16` the default layer holds 68,752 parameters, nearly all in a feed-forward layer 128 times wider than its input. With `dim_feedforward=64` it holds 3,280.

To classify a sequence, pool the final token vectors, for example the mean over real positions only, then apply a linear head. Including padding in the mean makes a short answer's prediction depend on how much padding it has.

Why it matters

A team's first transformer for open-ended answers trained slowly on a laptop and overfitted a few thousand answers. Each of its four layers had kept the default feed-forward width of 2,048 for a model width of 32, so almost all its weights sat in layers far wider than the data needed. Setting `dim_feedforward` to four times the model width cut the parameters by more than ten times, and validation loss improved.

Worked example: weak code

```
layer = nn.TransformerEncoderLayer(d_model=32, nhead=4)
encoder = nn.TransformerEncoder(layer, num_layers=4)
pooled = encoder(x).mean(dim=1)
```

1. **Line 1.** batch_first=False and dim_feedforward=2048: the batch is read as the sequence, and the layer is mostly feed-forward.
2. **Line 3.** No padding mask, and the mean includes padding positions.

Worked example: strong code

```
import torch
from torch import nn


class Block(nn.Module):

    """Pre-norm transformer block: attention, then feed-forward, each with
    a residual."""

    def __init__(self, d: int, heads: int) -> None:
```

```
 9         super().__init__()
10         self.norm_1 = nn.LayerNorm(d)
11         self.attn = nn.MultiheadAttention(d, heads, batch_first=True)
12         self.norm_2 = nn.LayerNorm(d)
(1)        self.ff = nn.Sequential(nn.Linear(d, 4 * d), nn.ReLU(),
       nn.Linear(4 * d, d))
14
15       def forward(self, x: torch.Tensor, is_pad: torch.Tensor) ->
       torch.Tensor:
16         h = self.norm_1(x)
(2)        x = x + self.attn(h, h, h, key_padding_mask=is_pad)[0]
(3)        return x + self.ff(self.norm_2(x))
19
20
21 def mean_over_real(h: torch.Tensor, is_pad: torch.Tensor) -> torch.Tensor:
22     """Mean of token vectors over non-padding positions, shape (batch,
       d)."""
23     keep = (~is_pad).unsqueeze(-1).float()
(4)    return (h * keep).sum(dim=1) / keep.sum(dim=1).clamp(min=1.0)
```

1. **Line 13.** Four times the model width, not a fixed 2,048.
2. **Line 17.** Residual around attention, on the normalised input, with padding ignored.
3. **Line 18.** Residual around the feed-forward layer.
4. **Line 24.** Averages real tokens only. clamp guards against a division by zero.

A block is attention then feed-forward, each with layer norm and a residual. Set batch_first, norm_first and dim_feedforward yourself, and pool over real tokens only.

Recap card

- **Pre-norm transformer block** x = x + attention(norm_1(x)); x = x + feed_forward(norm_2(x)).
- **Layer norm** Per token: subtract the mean, divide by sqrt(population variance + eps), then a learned scale and shift.
- **TransformerEncoderLayer defaults in PyTorch 2.15** batch_first=False, norm_first=False, dim_feedforward=2048, dropout=0.1. Set the first three yourself.
- **Pooling for classification** Mean over real positions only: sum with a 0/1 mask, divide by the count of real tokens.

Practice

Exercise 1. Multiple choice (level 1)

What do the residual connections, `x = x + sublayer(norm(x))`, give a deep transformer?

- a) Each sub-layer only adds a correction, and gradients reach early layers through the plain x path
- b) They double the number of parameters
- c) They replace the need for attention
- d) They make each token's vector sum to 1

Why?

- r1) The identity path passes the input and its gradient through unchanged, so a stack of many blocks still trains.
- r2) Addition is cheaper than multiplication.

Exercise 2. Predict the output (level 2)

What does this print? (PyTorch 2.15)

```
import torch
from torch import nn

out = nn.LayerNorm(4)(torch.tensor([[1.0, 2.0, 3.0, 4.0]]))
print([round(v, 4) for v in out[0].tolist()])
```

Exercise 3. Fill the gap (level 2)

Build a pre-norm encoder block that reads `(batch, length, features)`, with a feed-forward width of four times `d_model`.

```
layer = nn.TransformerEncoderLayer(64, 4, dim_feedforward=256,
    _____(1)=True, _____(2)=True)
```

Exercise 4. Write the function (level 3)

Write `layer_norm(x, eps=1e-5) -> np.ndarray`: normalise each vector along the last axis, as `nn.LayerNorm` does before its learned scale and shift.

- Return `(x - mean) / sqrt(var + eps)`, with the mean and the population variance (`np.var`) taken along the last axis.

What you need to know.

`arr.var(axis=-1)` returns the variance along the last axis, dividing by the count `n`: the population variance. `ddof=1` divides by `n - 1` instead:

```
np.array([1.0, 3.0]).var()          # 1.0
np.array([1.0, 3.0]).var(ddof=1)    # 2.0
```

Your code must pass these checks:

- Normalise each row along the last axis. A constant row becomes zeros, thanks to eps.
- Work along the last axis only, so each token is normalised by its own statistics.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Normalise along the last axis to mean 0 and standard deviation 1.
def layer_norm(
    x: np.ndarray,
    eps: float = 1e-05,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np


def layer_norm(x, eps=1e-5):
    return (x - x.mean()) / x.std()
```

Exercise 5. Write the function (level 3)

Write `mean_over_real(h, ids, pad_id=0) -> np.ndarray`: one vector per sequence, averaging only non-padding positions.

- `h` has shape `(N, L, d)` and `ids` has shape `(N, L)`. Return shape `(N, d)`.
- Raise `ValueError` when the shapes do not match, or a sequence is all padding.

Your code must pass these checks:

- Average only positions whose ID is not pad_id.
- Extra padding, whatever its vectors hold, must not change the result.
- Raise ValueError for mismatched shapes or an all-padding sequence.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Mean of token vectors over non-padding positions.
def mean_over_real(
    h: np.ndarray,
    ids: np.ndarray,
    pad_id: int = 0,
) -> np.ndarray: ...
```

Starter code:

```python
import numpy as np

```

```
3
4 def mean_over_real(h, ids, pad_id=0):
5     return h.mean(axis=1)
```

Exercise 6. Spot the bug (level 4)

The classifier trains, but its accuracy stays near chance. Each batch has shape `(32, 40, 64)`: batch, length, features. Tap the line responsible, then fix it.

```
1 from torch import nn
2
3 layer = nn.TransformerEncoderLayer(d_model=64, nhead=4,
    dim_feedforward=256, norm_first=True)
4 encoder = nn.TransformerEncoder(layer, num_layers=2)
5
6
7 def encode(x, is_pad):
8     return encoder(x, src_key_padding_mask=is_pad)
```

Write down the line number of the bug. Then write the fix.

Practical: Build a small transformer from scratch and prove each part works

Platform: Laptop. Work in `labs/transformer/` in this repository. You build attention, multi-head attention, sinusoidal positions and a pre-norm block in PyTorch, then train a small classifier on a synthetic task: does a sequence start and end with the same token? The task needs positions, and the tests check your attention against PyTorch's own, and that padding and the future cannot leak. You need PyTorch 2.15 on the laptop. When it trains, push the lab to your own GitHub repository with a written explanation: it is the roadmap's evidence for month 9, a transformer built from scratch.

1. Run `cd labs/transformer`, create a virtual environment, then `pip install -e .` and `pip install pytest`. Run `pytest`: most tests fail with NotImplementedError. Read `README.md` and every file in `src/tiny_transformer/`.
2. In `src/tiny_transformer/layers.py`, write `attention` (lessons B25-L3 and B25-L4). Run `pytest tests/test_layers.py -k attention`: it compares your output with `F.scaled_dot_product_attention`.
3. Write `sinusoidal_positions` and `MultiHeadSelfAttention` (lesson B25-L5), then `Block` (lesson B25-L6). Run `pytest tests/test_layers.py`.
4. Write `Classifier` in `src/tiny_transformer/model.py`: token embeddings plus positions, blocks, the mean over real tokens, and a linear head. Run `pytest`: every test passes.

5. Run `python -m tiny_transformer train`, then `python -m tiny_transformer train --no-positions`. Compare the accuracy and explain the gap.
6. Write your README: each part, the test that proves it, the two runs, and a diagram of one block. Push the folder to a new GitHub repository.

Sources

- PyTorch 2.15: nn.LayerNorm(4) maps [1, 2, 3, 4] to [-1.3416, -0.4472, 0.4472, 1.3416] and equals a NumPy (x - mean) / sqrt(var + 1e-5); TransformerEncoderLayer defaults are dim_feedforward 2048, dropout 0.1, activation relu, batch_first False, norm_first False, layer_norm_eps 1e-5; with d_model 16 the default layer has 68,752 parameters and 3,280 with dim_feedforward 64; a (batch, length, d) input without batch_first returns the same shape. (verified: Run on PyTorch 2.15.0 (PyPI wheel, CPU), November 2026.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B25-L1 Tokens, vocabularies and embeddings

B25-L1 exercise 1

Hints. (1) What happens to a word the vocabulary lacks? (2) Which special token fills short sequences? (3) Unseen words need an ID.

Answer. a) So that words not seen in training map to one known ID instead of raising an error Because: r1) New text always contains words the training text lacked, and every token needs a valid ID.

Every token must map to a row of the embedding table. `<unk>` catches every word the training text never contained.

Common wrong answers.

- `b`: <unk> gives every unseen word the same ID. It adds no weight.
- `c`: An end marker is a separate special token. <unk> stands for words missing from the vocabulary.
- `d`: That is <pad>, ID 0 here.
- `a|r2`: Right answer, wrong reason. It is about giving every token a valid ID.

B25-L1 exercise 2

Hints. (1) Input shape (2, 4) of IDs. (2) Each ID becomes a row of length embedding_dim. (3) What does padding_idx do to its row?

Answer.

```
1 (2, 4, 4) True
```

Two sequences of four IDs become two sequences of four 4-value vectors. The padding row starts at zero and stays there.

Common wrong answers.

- `(2, 4) True`: Each ID becomes a vector of embedding_dim values, so a dimension of 4 is added.
- `(2, 4, 10) True`: The last dimension is embedding_dim, 4. The 10 is the number of rows in the table.
- `(2, 4, 4) False`: With padding_idx=0, row 0 starts at zero.

B25-L1 exercise 3

Hints. (1) Which argument names the ID that never trains? (2) Which dtype do indices use? (3) `padding_idx` and `long`.

Answer.

```
emb = nn.Embedding(len(vocab), 32, padding_idx=0)
vectors = emb(torch.tensor(batch_ids, dtype=torch.long))
```

`padding_idx=0` keeps row 0 at zero, and `torch.long` IDs index the table.

Common wrong answers.

- g1 = `padding_idx`; g2 = `float32`: Embedding looks up rows by index, so IDs must be integers: torch.long.

B25-L1 exercise 4

Hints. (1) Check the arguments, then `text.lower().split()`. (2) `vocab.get(word, vocab["<unk>"])`, then slice `[:length]`. (3) Add `[vocab["<pad>"]] * (length - len(ids))`.

Answer.

```
def encode(text: str, vocab: dict[str, int], length: int) -> list[int]:
    """Token IDs for one text, truncated or padded to length.

    Args:
        text: The raw text.
        vocab: Word to ID, including "<pad>" and "<unk>".
        length: Number of IDs to return.

    Returns:
        Exactly length IDs.

    Raises:
        ValueError: If length is below 1 or the special tokens are
    missing.
    """
    if length < 1 or "<pad>" not in vocab or "<unk>" not in vocab:
        raise
    ValueError("need length >= 1 and a vocabulary with <pad> and <unk>")
    ids = [vocab.get(word, vocab["<unk>"]) for word in
    text.lower().split()][:length]
    return ids + [vocab["<pad>"]] * (length - len(ids))
```

Lower-casing matches the vocabulary's spelling, `get` with a default handles unseen words, and truncating before padding gives exactly `length` IDs.

B25-L1 exercise 5

Hints. (1) Check `ids.dtype.kind` is `"i"` or `"u"`. (2) Check the smallest and largest ID against `len(table)`. (3) Return `table[ids]`.

Answer.

```python
import numpy as np


def embed(ids: np.ndarray, table: np.ndarray) -> np.ndarray:
    """Look up embedding rows by integer ID, as nn.Embedding does.

    Args:
        ids: Integer IDs of any shape.
        table: Embedding table, shape (vocab_size, dim).

    Returns:
        Array of shape ids.shape + (dim,).

    Raises:
        ValueError: If ids are not integers or fall outside the table.
    """
    if ids.dtype.kind not in "iu":
        raise ValueError(f"IDs must be integers, got {ids.dtype}")
    if ids.size and (ids.min() < 0 or ids.max() >= len(table)):
        raise ValueError(f"IDs must lie in 0 to {len(table) - 1}")
    return table[ids]
```

Indexing the table with an integer array returns one row per ID. In NumPy a negative index counts from the end, so -1 would silently return the last row: the range check stops that.

B25-L1 exercise 6

Hints. (1) Which IDs do the first two sorted words get? (2) Which special tokens already use those IDs? (3) enumerate(words, start=2).

Answer. The bug is on line 3.

Fixed version:

```python
def build_vocab(train_texts):
    words = sorted({word for text in train_texts for word in
    text.lower().split()})
    vocab = {word: i for i, word in enumerate(words, start=2)}
    vocab["<pad>"] = 0
    vocab["<unk>"] = 1
    return vocab
```

`enumerate` starts at 0, so the first two words alphabetically share IDs with `<pad>` and `<unk>`. Start at 2 so special tokens keep their own IDs.

Common wrong answers.

- Line 4: <pad> should be 0. The trouble is which words also got 0 and 1.
- Line 2: Sorting the distinct words makes IDs repeatable. That is fine.

B25-L2 Sequence models, recurrence and its limits

B25-L2 exercise 1

Hints. (1) How does the gradient reach step 1 from step 200? (2) What happens to a product of many numbers below 1? (3) Vanishing gradients.

Answer. a) The gradient reaching that word is multiplied by a factor at every step, and products of factors below 1 shrink towards zero Because: r1) Training signal for early words passes back through every step, so it fades: 0.9 over 50 steps is 0.0052.

The gradient for an early word is a product over every later step. With factors below 1 it vanishes, so early words barely train.

Common wrong answers.

- `b`: An RNN can run over any length. The problem is how faintly early steps are trained.
- `c`: The embedding table is the same for every position. The fading happens in the recurrence.
- `d`: Nothing resets the state. It carries on, but early information fades.
- `a|r2`: Right answer, wrong reason. It is about how the gradient travels, not what words matter.

B25-L2 exercise 2

Hints. (1) batch_first=True: (batch, length, features). (2) Outputs: one hidden state per step. (3) h and c: (layers, batch, hidden).

Answer.

```
1 (2, 5, 8) (1, 2, 8) (1, 2, 8)
```

Outputs give a hidden state of size 8 for each of 5 steps in 2 sequences. The final h and c add a layer dimension in front.

Common wrong answers.

- `(2, 5, 8) (2, 8) (2, 8)`: h and c keep a leading dimension for the number of layers: (1, 2, 8).
- `(2, 5, 4) (1, 2, 8) (1, 2, 8)`: Outputs hold the hidden state, so their last dimension is hidden_size, 8.

B25-L2 exercise 3

Hints. (1) Which argument puts batch first? (2) Which function squashes values into (-1, 1)? (3) `batch_first` and `tanh`.

Answer.

```python
lstm = nn.LSTM(32, 64, batch_first=True)

h = np.tanh(x_t @ W_ih.T + h @ W_hh.T + b)
```

`batch_first=True` matches the usual data layout, and `tanh` is the RNN's default non-linearity.

Common wrong answers.

- g1 = `batch_first`; g2 = `maximum`: nn.RNN's default non-linearity is tanh, which keeps the state between -1 and 1.

B25-L2 exercise 4

Hints. (1) Check the three shapes first. (2) Start with `h = np.zeros(len(b))` before the loop. (3) In the loop, update h from x_t and the previous h, and append it.

Answer.

```python
import numpy as np


def rnn_forward(x: np.ndarray, W_ih: np.ndarray, W_hh: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Hidden states of a tanh RNN over one sequence, as nn.RNN computes them.

    Args:
        x: One sequence, shape (length, input_size).
        W_ih: Input weights, shape (hidden_size, input_size).
        W_hh: Recurrent weights, shape (hidden_size, hidden_size).
        b: Combined bias, shape (hidden_size,).

    Returns:
        Hidden state after each step, shape (length, hidden_size).

    Raises:
        ValueError: If the shapes do not fit together.
    """
    if x.ndim != 2 or W_ih.shape[1] != x.shape[1] or W_hh.shape != (len(b), len(b)):
        raise ValueError(f"shapes do not fit: x {x.shape}, W_ih {W_ih.shape}, W_hh {W_hh.shape}")
    h = np.zeros(len(b))
    states = []
    for x_t in x:
        h = np.tanh(x_t @ W_ih.T + h @ W_hh.T + b)
        states.append(h)
    return np.stack(states) if states else np.zeros((0, len(b)))
```

The state is created once and updated at every step, so each output depends on everything before it. That dependence is also why the steps cannot run in parallel.

B25-L2 exercise 5

Hints. (1) 0.9 ** 10 is about 0.35. (2) Raise that to the fifth power. (3) 1.1 ** 50 grows the same way, upwards.

Answer.

```
0.0052 117.4
```

A product of 50 factors of 0.9 vanishes to 0.0052, and of 1.1 explodes to 117.4. That is why plain RNNs lose long-range signal or become unstable.

Common wrong answers.

- `0.0 inf`: Fifty steps is not enough to reach exactly 0 or infinity, but the signal is already a two-hundredth, or over a hundred times too large.
- `0.45 5.0`: That would be adding 50 small changes. Each step multiplies.

B25-L2 exercise 6

Hints. (1) What layout does nn.LSTM expect by default? (2) With that layout, what does dimension 0 of (64, 30, 32) mean? (3) batch_first=True.

Answer. The bug is on line 8.

Fixed version:

```
from torch import nn


class AnswerClassifier(nn.Module):
    def __init__(self, vocab_size, n_classes):
        super().__init__()
        self.tokens = nn.Embedding(vocab_size, 32, padding_idx=0)
        self.lstm = nn.LSTM(32, 64, batch_first=True)
        self.head = nn.Linear(64, n_classes)

    def forward(self, ids):
        outputs, (h, c) = self.lstm(self.tokens(ids))
        return self.head(h[-1])
```

By default, the LSTM reads `(64, 30, 32)` as 30 sequences of length 64, running across different answers. There is no error, because the shapes still fit. `batch_first=True` makes it run along each answer.

Common wrong answers.

- Line 13: h[-1] is the last layer's final state, a fine summary. Which axis did the LSTM treat as time?

- Line 7: The embedding is right. Its (64, 30, 32) output is read wrongly on the next layer.

B25-L3 Scaled dot-product attention

B25-L3 exercise 1

Hints. (1) How does the spread of a dot product change with d? (2) What does softmax do with huge inputs? (3) Keep scores near unit scale.

Answer. a) Dot products of d-dimensional vectors grow with d. Large scores saturate softmax and its gradients vanish Because: r1) The standard deviation of a dot product of unit-variance vectors is about sqrt(d), so dividing by it keeps scores near unit scale.

Dividing by `sqrt(d)` cancels the growth of the dot product with width, so softmax stays in a range with useful gradients.

Common wrong answers.

- `b`: Softmax makes rows sum to 1, scaled or not.
- `c`: Scaling reduces typical scores but does not rule out overflow. Subtracting the row maximum does.
- `d`: The scale applies to the scores, not to V.
- `a|r2`: Right answer, wrong reason. It is about keeping scores at a scale where softmax trains.

B25-L3 exercise 2

Hints. (1) Scores: 1/sqrt(2), 0, 1/sqrt(2). (2) exp of 0.707 is about 2.03. (3) Normalise 2.03, 1, 2.03.

Answer.

```
1 [0.4011, 0.1978, 0.4011] 20.0
```

The first and third keys match the query equally, so they share most of the weight. The weighted average of 10, 20 and 30 is 20, since the weights are symmetric.

Common wrong answers.

- `[0.5, 0.0, 0.5] 20.0`: Softmax never gives exactly 0. The second key scores 0, and exp(0) is 1, not 0.
- `[0.3333, 0.3333, 0.3333] 20.0`: The second key scores lower than the other two, so it gets less weight.

B25-L3 exercise 3

Hints. (1) Scale by the square root of the width. (2) Which value, subtracted, makes the largest exponent 0? (3) `sqrt` and `max`.

Answer.

```
scores = Q @ K.T / np.sqrt(d)
e = np.exp(scores - scores.max(axis=-1, keepdims=True))
```

`np.sqrt(d)` scales the scores, and subtracting each row's `max` makes every exponent at most 0.

Common wrong answers.

- g1 = `sqrt`; g2 = `mean`: Subtracting the mean can still leave large positive values. The maximum makes the largest exponent 0.

B25-L3 exercise 4

Hints. (1) `x.max(axis=axis, keepdims=True)`. (2) Exponentiate the shifted values. (3) Divide by `e.sum(axis=axis, keepdims=True)`.

Answer.

```
import numpy as np


def softmax(x: np.ndarray, axis: int = -1) -> np.ndarray:
    """Softmax along an axis, with the maximum subtracted for stability.

    Args:
        x: Scores.
        axis: Axis whose values become probabilities.

    Returns:
        Array of x's shape, summing to 1 along axis.
    """
    shifted = x - x.max(axis=axis, keepdims=True)
    e = np.exp(shifted)
    return e / e.sum(axis=axis, keepdims=True)
```

Shifting by the maximum leaves the result unchanged and keeps every exponent at most 1. Using the same axis with `keepdims` makes each row, not the whole array, sum to 1.

B25-L3 exercise 5

Hints. (1) Check `Q.shape[-1] == K.shape[-1]` and `len(K) == len(V)`. (2) `softmax(Q @ K.T / np.sqrt(Q.shape[-1]), axis=-1)`. (3) Return `weights @ V` and the weights.

Answer.

```python
import numpy as np


def attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Scaled dot-product attention for one sequence.

    Args:
        Q: Queries, shape (n_queries, d).
        K: Keys, shape (n_keys, d).
        V: Values, shape (n_keys, d_v).

    Returns:
        Outputs (n_queries, d_v) and weights (n_queries, n_keys).

    Raises:
        ValueError: If Q and K differ in width or K and V in length.
    """
    if Q.shape[-1] != K.shape[-1] or len(K) != len(V):
        raise ValueError(f"shapes do not fit: Q {Q.shape}, K {K.shape}, V {V.shape}")
    weights = softmax(Q @ K.T / np.sqrt(Q.shape[-1]), axis=-1)
    return weights @ V, weights
```

Each query's row of weights covers all keys and sums to 1, and the output mixes the values with those weights. This is the computation `scaled_dot_product_attention` performs.

B25-L3 exercise 6

Hints. (1) Which axis of the scores runs over keys? (2) Which axis does the sum use? (3) axis=-1.

Answer. The bug is on line 8.

Fixed version:

```python
import numpy as np


def attention(Q, K, V):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    weights = weights / weights.sum(axis=-1, keepdims=True)
    return weights @ V, weights
```

Rows are queries and columns are keys. Each query's weights must sum to 1 across the keys, so normalise along axis -1. Summing along axis 0 mixes different queries.

Common wrong answers.

- Line 6: Subtracting each row's maximum is right. The normalisation uses a different axis.
- Line 5: The scaled scores are right. Look at how the weights are normalised.

B25-L4 Masks for padding and for the future

B25-L4 exercise 1

Hints. (1) What is the target for position i? (2) Where in the sequence is that target? (3) Hide the future.

Answer. a) A causal mask, so position i sees positions 0 to i only, plus masking of padding Because: r1) Without the causal mask, each position can read the word it must predict, and the model learns to copy it.

The next word sits at position i + 1. A causal mask keeps every position from attending to it.

Common wrong answers.

- `b`: Padding is not the only leak. Each position can still read later words.
- `c`: Copying the next word is the easiest way to minimise the loss. Nothing stops it.
- `d`: The current word is input. The next word is the target, and it lies in the future.
- `a|r2`: Right mask, wrong reason. It stops the model reading its own answers.

B25-L4 exercise 2

Hints. (1) Which entries does tril keep? (2) Row 0 allows one key, row 1 two, row 2 three. (3) Equal scores share the weight evenly.

Answer.

```
1 [[1.0, 0.0, 0.0], [0.5, 0.5, 0.0], [0.3333, 0.3333, 0.3333]]
```

Each row spreads its weight evenly over the keys it may see: one, then two, then three.

Common wrong answers.

- `[[0.3333, 0.3333, 0.3333], [0.3333, 0.3333, 0.3333], [0.3333, 0.3333, 0.3333]]`: The mask sets future scores to minus infinity, so their weights are 0.

- `[[0.3333, 0.3333, 0.3333], [0.0, 0.5, 0.5], [0.0, 0.0, 1.0]]`: tril keeps the lower triangle: row i allows columns 0 to i.

B25-L4 exercise 3

Hints. (1) Lower or upper triangle? (2) What score makes exp return exactly 0? (3) `tril` and `inf`.

Answer.

```
allowed = np.tril(np.ones((L, L), dtype=bool))
scores = np.where(allowed, scores, -np.inf)
```

`np.tril` keeps positions 0 to i in row i, and minus infinity turns masked scores into weights of 0.

Common wrong answers.

- g1 = `triu`; g2 = `inf`: triu keeps the upper triangle, which lets each position see the future and not the past.

B25-L4 exercise 4

Hints. (1) Check the mask's shape and `allowed.any(axis=1).all()`. (2) `np.where(allowed, scores, -np.inf)` before the softmax. (3) Subtract the row maximum, exponentiate, normalise, multiply by V.

Answer.

```
import numpy as np


def masked_attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray, allowed: np.ndarray) -> np.ndarray:
    """Scaled dot-product attention where allowed is False gets zero weight.

    Args:
        Q: Queries, shape (n_queries, d).
        K: Keys, shape (n_keys, d).
        V: Values, shape (n_keys, d_v).
        allowed: Boolean, shape (n_queries, n_keys). True where attention may look.

    Returns:
        Outputs, shape (n_queries, d_v).

    Raises:
        ValueError: If the mask has the wrong shape or a row allows no key.
    """
    if allowed.shape != (len(Q), len(K)):
```

```
        raise ValueError(f"mask {allowed.shape} does not match ({len(Q)}, {len(K)})")
    if not allowed.any(axis=1).all():
        raise ValueError("every query needs at least one allowed key")
    scores = np.where(allowed, Q @ K.T / np.sqrt(Q.shape[-1]), -np.inf)
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    return (weights / weights.sum(axis=-1, keepdims=True)) @ V
```

A score of 0 is an ordinary score, not a mask: it still gets weight. Minus infinity gives exp of 0 exactly, so masked keys contribute nothing, and the guards rule out rows that would become nan.

B25-L4 exercise 5

Hints. (1) Validate first: 1-D, non-empty, first token not padding. (2) `np.tril(np.ones((L, L), dtype=bool))` for the causal part. (3) AND it with `(ids != pad_id)[None, :]`, which spreads across the columns.

Answer.

```
import numpy as np


def causal_padding_mask(ids: np.ndarray, pad_id: int = 0) -> np.ndarray:
    """Allowed attention for one right-padded sequence: past or present, never padding.

    Args:
        ids: Token IDs, shape (length,), padding only at the end.
        pad_id: The padding ID.

    Returns:
        Boolean (length, length): True where query i may attend to key j.

    Raises:
        ValueError: If ids is not 1-D, is empty, or starts with padding.
    """
    if ids.ndim != 1 or len(ids) == 0 or ids[0] == pad_id:
        raise ValueError("need a non-empty 1-D sequence that starts with a real token")
    causal = np.tril(np.ones((len(ids), len(ids)), dtype=bool))
    return causal & (ids != pad_id)[None, :]
```

Columns are keys, so the padding test is shaped `(1, L)` and spreads down every row. Padding rows still see the real tokens before them, so no row is empty.

B25-L4 exercise 6

Hints. (1) In nn.MultiheadAttention, what does True in key_padding_mask mean? (2) Which positions does real mark True? (3) Pass ~real.

Answer. The bug is on line 8.

Fixed version:

```
1 from torch import nn
2
3 attn = nn.MultiheadAttention(embed_dim=64, num_heads=4, batch_first=True)
4
5
6 def encode(x, ids):
7     real = ids != 0
8     out, _ = attn(x, x, x, key_padding_mask=~real)
9     return out
```

`key_padding_mask` ignores keys marked `True`. Passing `real` ignores every word and attends only to padding. Pass `~real`, or `ids == 0`.

Common wrong answers.

- Line 7: real correctly marks the text positions. The question is what key_padding_mask expects.
- Line 3: batch_first=True is right for (batch, length, features) inputs.

B25-L5 Multi-head attention and position information

B25-L5 exercise 1

Hints. (1) What does attention compare? (2) What happens to outputs if you shuffle the inputs? (3) Order is invisible without positions.

Answer. a) Attention compares contents only, so without positions a shuffled sentence gives the same outputs, shuffled Because: r1) Without positions, "not good" and "good not" are the same set of vectors to attention.

Attention is a function of contents. Adding a position vector to each token makes the same word at different places look different.

Common wrong answers.

- `b`: Positions are added, so the width stays d.
- `c`: Padding is handled by a mask (lesson B25-L4), not by positions.
- `d`: Softmax accepts any real inputs.
- `a|r2`: Right answer, wrong reason. It is about word order, not speed.

B25-L5 exercise 2

Hints. (1) reshape gives (2, 5, 2, 4). Then swap dimensions 1 and 2. (2) Parameters: 4 * d * d + 4 * d with d = 8. (3) Does the head count appear in that formula?

Answer.

```
(2, 2, 5, 4) 288 288
```

Heads split the width, so the four projections stay 8 by 8 with biases: 288 parameters for any head count that divides 8.

Common wrong answers.

- `(2, 5, 2, 4) 288 288`: transpose(1, 2) swaps the length and head dimensions: (N, h, L, d // h).
- `(2, 2, 5, 4) 288 576`: More heads split the same d into smaller pieces. The projections stay d by d.

B25-L5 exercise 3

Hints. (1) Which method swaps two dimensions? (2) Which argument puts batch first? (3) `transpose` and `batch_first`.

Answer.

```
heads = x.reshape(N, L, h, d // h).transpose(1, 2)
attn = nn.MultiheadAttention(64, 4, batch_first=True)
```

`transpose(1, 2)` gives `(N, h, L, d // h)`, so each head attends over the full length. `batch_first=True` matches the usual layout.

Common wrong answers.

- g1 = `reshape`; g2 = `batch_first`: reshape cannot move the head dimension in front of the length. transpose(1, 2) swaps them.

B25-L5 exercise 4

Hints. (1) Check `x.shape[1] % n_heads` first. (2) Reshape to `(length, n_heads, d // n_heads)`. (3) Then `transpose(1, 0, 2)` to put heads first.

Answer.

```
import numpy as np


def split_heads(x: np.ndarray, n_heads: int) -> np.ndarray:
    """Split the width of one sequence into attention heads.

    Args:
        x: Shape (length, d).
        n_heads: Number of heads. d must divide by it.
```

```
    Returns:
        Shape (n_heads, length, d // n_heads).

    Raises:
        ValueError: If the shapes do not allow the split.
    """
    if x.ndim != 2 or n_heads < 1 or x.shape[1] % n_heads:
        raise ValueError(f"cannot split shape {x.shape} into {n_heads} heads")
    length, d = x.shape
    return x.reshape(length, n_heads, d // n_heads).transpose(1, 0, 2)
```

Reshaping to `(L, h, dh)` cuts each row into consecutive column blocks, and the transpose gathers each block across rows. Reshaping straight to `(h, L, dh)` splits the rows instead, mixing tokens between heads.

B25-L5 exercise 5

Hints. (1) `pos` as a column: `np.arange(n)[:, None]`; `i` as a row: `np.arange(0, d, 2)[None, :]`. (2) `angle = pos / 10000 ** (i / d)`, shape (n, d // 2). (3) Fill `pe[:, 0::2]` with sin and `pe[:, 1::2]` with cos.

Answer.

```
import numpy as np


def sinusoidal_positions(n: int, d: int) -> np.ndarray:
    """Sinusoidal position encodings.

    Args:
        n: Number of positions.
        d: Width, a positive even number.

    Returns:
        Shape (n, d): sine in even columns, cosine in odd columns.

    Raises:
        ValueError: If n is below 1 or d is not positive and even.
    """
    if n < 1 or d < 2 or d % 2:
        raise ValueError(f"need n >= 1 and an even d >= 2, got {n} and {d}")
    pos = np.arange(n)[:, None]
    angle = pos / 10000 ** (np.arange(0, d, 2)[None, :] / d)
    pe = np.zeros((n, d))
    pe[:, 0::2] = np.sin(angle)
    pe[:, 1::2] = np.cos(angle)
    return pe
```

Each pair of columns is a sine and cosine at one frequency, from fast in the first pair to slow in the last. Broadcasting a column of positions against a row of exponents builds the whole table at once.

B25-L5 exercise 6

Hints. (1) What axis order does split_heads produce? (2) reshape reads values in memory order: heads first, then rows. (3) Transpose back to (L, h, dh) before reshaping.

Answer. The bug is on line 6.

Fixed version:

```
1 import numpy as np
2
3
4 def merge_heads(heads):
5     n_heads, length, dh = heads.shape
6     return heads.transpose(1, 0, 2).reshape(length, n_heads * dh)
```

`reshape` takes values in their current order, all of head 0 first, so rows of the result mix tokens. Transposing to `(length, n_heads, dh)` first puts each token's heads next to each other, and the reshape then rebuilds its vector.

Common wrong answers.

- Line 5: Reading the three sizes is right. The data is reshaped in the wrong order.

B25-L6 The transformer block, residuals, layer norm and feed-forward

B25-L6 exercise 1

Hints. (1) What does x + f(x) equal when f outputs zeros? (2) How does a gradient pass through an addition? (3) Corrections, and a clean gradient path.

Answer. a) Each sub-layer only adds a correction, and gradients reach early layers through the plain x path Because: r1) The identity path passes the input and its gradient through unchanged, so a stack of many blocks still trains.

With `x + f(x)`, a block starts close to the identity and learns a correction, and the gradient flows back through the `x` path without shrinking.

Common wrong answers.

- `b`: An addition has no parameters.
- `c`: The residual wraps attention. Attention still does the mixing between tokens.
- `d`: That describes softmax weights, not residuals.
- `a|r2`: Right answer, wrong reason. It is about the clean path for inputs and gradients.

B25-L6 exercise 2

Hints. (1) Mean 2.5. (2) Population variance 1.25, standard deviation 1.118. (3) (1 - 2.5) / 1.118.

Answer.

```
[-1.3416, -0.4472, 0.4472, 1.3416]
```

Subtract the mean, 2.5, and divide by the standard deviation over the four values, 1.118. A new layer's learned scale is 1 and shift 0, so the result is unchanged by them.

Common wrong answers.

- `[0.1, 0.2, 0.3, 0.4]`: Layer norm does not divide by the sum. It subtracts the mean and divides by the standard deviation.
- `[-1.1619, -0.3873, 0.3873, 1.1619]`: Those use the sample standard deviation (n - 1). LayerNorm divides by the population standard deviation (n).

B25-L6 exercise 3

Hints. (1) One argument fixes the layout. (2) One argument moves the layer norm before each sub-layer. (3) `batch_first` and `norm_first`.

Answer.

```
layer = nn.TransformerEncoderLayer(64, 4, dim_feedforward=256,
    batch_first=True, norm_first=True)
```

`batch_first=True` reads `(batch, length, d)`, and `norm_first=True` gives the pre-norm block.

Common wrong answers.

- g1 = `batch_first`; g2 = `batch_first`: Both settings are needed: batch_first for the layout and norm_first for the pre-norm order.

B25-L6 exercise 4

Hints. (1) `x.mean(axis=-1, keepdims=True)`. (2) `x.var(axis=-1, keepdims=True)`: population variance by default. (3) Divide by `np.sqrt(var + eps)`.

Answer.

```python
import numpy as np


def layer_norm(x: np.ndarray, eps: float = 1e-5) -> np.ndarray:
    """Normalise along the last axis to mean 0 and standard deviation 1.

    Args:
        x: Array whose last axis holds features.
        eps: Added to the variance to avoid dividing by zero.

    Returns:
        Array of x's shape.
    """
    mean = x.mean(axis=-1, keepdims=True)
    var = x.var(axis=-1, keepdims=True)
    return (x - mean) / np.sqrt(var + eps)
```

Statistics along the last axis normalise each token on its own, which is why layer norm does not depend on the batch. `eps` turns a constant vector into zeros instead of a division by zero.

B25-L6 exercise 5

Hints. (1) A weight of 1 for real positions and 0 for padding: `(ids != pad_id)[:, :, None]`. (2) Sum `h * keep` over axis 1, and count the real positions. (3) Check no count is 0, then divide.

Answer.

```python
import numpy as np


def mean_over_real(h: np.ndarray, ids: np.ndarray, pad_id: int = 0) ->
    np.ndarray:
    """Mean of token vectors over non-padding positions.

    Args:
        h: Token vectors, shape (N, L, d).
        ids: Token IDs, shape (N, L).
        pad_id: The padding ID.

    Returns:
        Shape (N, d).

    Raises:
```

```
        ValueError: If the shapes do not match or a sequence is all
    padding.
    """
    if h.shape[:2] != ids.shape:
        raise ValueError(f"h {h.shape} and ids {ids.shape} do not match")
    keep = (ids != pad_id)[:, :, None].astype(float)
    counts = keep.sum(axis=1)
    if (counts == 0).any():
        raise ValueError("a sequence has no real tokens")
    return (h * keep).sum(axis=1) / counts
```

Zeroing padding is half the job. Dividing by the number of real tokens, not the padded length, makes the result independent of how much padding a batch adds.

B25-L6 exercise 6

Hints. (1) What layout does TransformerEncoderLayer assume by default? (2) With that default, what does dimension 0 of (32, 40, 64) mean? (3) batch_first=True.

Answer. The bug is on line 3.

Fixed version:

```
from torch import nn

layer = nn.TransformerEncoderLayer(d_model=64, nhead=4,
    dim_feedforward=256, norm_first=True, batch_first=True)
encoder = nn.TransformerEncoder(layer, num_layers=2)


def encode(x, is_pad):
    return encoder(x, src_key_padding_mask=is_pad)
```

Without `batch_first=True`, the layer treats the 32 answers as positions and the 40 positions as the batch, so attention mixes different answers. The output shape is unchanged, so nothing fails.

Common wrong answers.

- Line 4: Stacking two layers is fine. The layer they copy reads the wrong layout.
- Line 8: The padding mask is passed correctly. The layer misreads the batch.